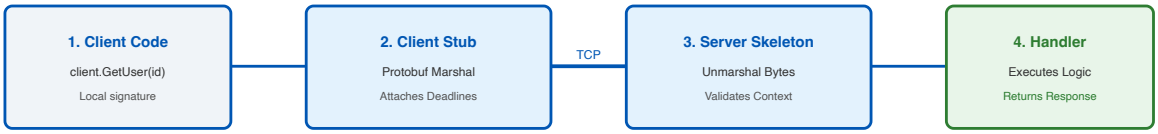
Introduction to RPC: Protocols, Stubs & Distributed Primitives

Decomposition of Local Call Abstraction, Binary Marshaling, gRPC, and HTTP/2 Multiplexing

EXECUTIVE SUMMARY

Remote Procedure Calls (RPC) allow microservices to invoke functions across network boundaries as if they were local in-memory calls. However, treating a network boundary as an in-memory pointer dereference creates critical failure modes. High-performance RPC relies on strict IDL typing, compact binary serialization (Protobuf), and multiplexed transport (HTTP/2).

1. THE RPC INVOCATION LIFECYCLE



2. THE LEAKY ABSTRACTION: LOCAL VS REMOTE CALLS

Property	Local Function Call	Remote Procedure Call
Latency	~10 - 20 nanoseconds (10^{-8} s).	~0.5 - 50 milliseconds (10^{-3} s) (50,000x slower).
Memory Boundary	Shared heap and stack pointer.	Disjoint memory; must marshal to bytes.
Failure Semantics	Deterministic: either executes or crashes process.	Partial failure: network drops response after execution.

3. BINARY SERIALIZATION: PROTOCOL BUFFERS (.PROTO)

Protocol Buffers encode structured data into compact binary payloads using tag-length-value (TLV) varint encoding:

```
// Protobuf Interface Definition Language (IDL)
syntax = "proto3";
package payments.v1;

service PaymentService {
  rpc ProcessPayment (PaymentRequest) returns (PaymentResponse);
}

message PaymentRequest {
  string transaction_id = 1;  // Tag 1 (1 byte overhead)
  int64 amount_cents     = 2;  // Tag 2 (varint encoding)
  string currency        = 3;  // Tag 3
}
```

4. PROTOCOL BUFFERS VS. JSON BENCHMARK COMPARISON

Wire Size: -70%

Protobuf strips field name strings from payloads, transmitting only compact integer field tags (1, 2, 3).

CPU Cost: 5x Faster

Binary decoding skips expensive string scanning, quote escaping, and float conversions used in JSON parsers.

Compile Typing

Compiler generates strict Go, Java, or C++ stubs. Schema regressions are caught at compile-time in CI.

5. HTTP/2 TRANSPORT PRIMITIVES IN GRPC

HTTP/2 Primitive	Operating Mechanism	Microservice Benefit
Multiplexing	Interleaves multiple request/response streams across a single persistent TCP connection.	Eliminates TCP 3-way handshakes and connection pool limits.
HPACK Compression	Maintains differential header compression table between client and server.	Slashes repeated Auth token / Tracing header overhead by 80%.
Binary Framing	Encapsulates data into discrete HEADERS and DATA binary frames.	Fast machine parsing; eliminates delimiter ambiguity.

6. THE FOUR GRPC STREAMING ARCHETYPES

1. Unary RPC

```
rpc GetUser (UserRequest) returns (UserResponse);
```

Standard point-to-point request/response query.

2. Server Streaming

```
rpc StreamPrices (Market) returns (stream Price);
```

Client sends subscription; server pushes updates.

3. Client Streaming

```
rpc UploadTelemetry (stream Metric) returns (Ack);
```

Client streams telemetry chunks; server summarizes.

4. Bidirectional Streaming

```
rpc Chat (stream Msg) returns (stream Msg);
```

Full-duplex independent concurrent messaging.

7. CANONICAL STATUS CODES & CONTEXT PROPAGATION

Unlike ambiguous HTTP status codes, gRPC enforces 16 canonical error codes:

```
// Propagating Deadlines and Context in Go
ctx, cancel := context.WithTimeout(context.Background(), 250*time.Millisecond)
defer cancel()

resp, err := client.ProcessPayment(ctx, req)
if err != nil {
    st, ok := status.FromError(err)
    if ok && st.Code() == codes.DeadlineExceeded {
        log.Warn("Downstream payment service timed out after 250ms budget")
    }
}
```

8. PRODUCTION RPC VERIFICATION CHECKLIST

RPC Engineering Standards

1. **Never Reuse Tag Numbers:** Once a field tag number is assigned in `.proto`, mark it `reserved` if deprecated.
2. **Client-Side Load Balancing:** Use round-robin DNS or Envoy mesh sidecars to avoid pod pinning on persistent HTTP/2 connections.
3. **Deadline Propagation:** Ensure servers verify `ctx.Err() == context.Canceled` before executing heavy SQL.
4. **Rich Error Models:** Standardize on `google.rpc.Status` with typed protobuf error details.